Dgrammar and DPGrammar: Frontier Constrained Decoding for Diffusion Language Models

Jeng-Yue Liu[1], Wilson Zheng[1], Haoling Pu[1]
[1]Language Technologies Institute, Carnegie Mellon University
{buffettl,wilsonz,haolingp}@andrew.cmu.edu

Abstract

Structured outputs from language models (valid JSON under a schema, syntactic code, and similar targets) are safety and integration critical, yet discrete diffusion language models (dLLMs) update many positions in parallel and lack a monotonic prefix, which makes standard autoregressive constrained decoding ill suited. We argue that both *reliable* constraints and *measurable* constraint side cost matter for practical deployment. Our intervention has two parts. **Dgrammar** is a deterministic decoding stack built on LLGUIDANCE (**?**): hard masks only at the decoding frontier, incremental `LLMatcher` state, AIMD style adaptive batching, violator focused remasking with in place resampling from the same forward when possible, early termination when the grammar accepts, and asynchronous overlap of `compute_mask` with the next GPU forward. This targets the stochastic lookahead and positionwise verification limits of the natural baseline, LAVE (**?**). **DPGrammar** extends the same driver with `dp_fix_prefix`, a Viterbi style dynamic program that jointly reassigns a violated contiguous segment under one forward's log probabilities when greedy repair is myopic. On `jsb_medium`, our reimplementation of LAVE reaches 68.8% schema validity (n=586, nine shards), while Dgrammar reaches 84.9% and DPGrammar 99.2% on a merged nine shard evaluation (n=511), with DPGrammar reducing mean resamples per instance from 32.43 to 1.06 at the cost of occasional long tail latency. Broadly, the work shows that parser aligned frontier decoding plus explicit timing instrumentation is a strong foundation, and that segment level DP repair can further close validity gaps left by greedy correction.

1 Introduction

1.1 Problem and significance

Many applications require model outputs to obey a formal grammar or schema: APIs that consume JSON, tools that compile code, and pipelines that validate structured records. Diffusion language models such as LLaDA (**?**) are attractive because they can refine many tokens under bidirectional context, but they do not offer a single left to right chain on which classical FSM style masking is defined. Ensuring constraints without destroying the model's joint proposals, and without hiding constraint cost inside a single wall clock number, is an open engineering and research problem.

1.2 Research gap

Recent work on constrained *diffusion* decoding includes CFG style enforcement (**?**), lookahead then verify decoding (LAVE) (**?**), and heavier dynamic programming style inference (**?**). LAVE is a strong reference: it samples masked suffixes to test whether a candidate token can extend to a valid completion. Gaps we target are (i) *stochastic* false rejection and cost that grows with sample count, (ii) verification that is not always aligned with a single incremental parser state tied to the actual tensor, and (iii) limited reporting of *per component* constraint time as compared to forward time, which matters when grammar checks are frequent.

1.3 Approach and how it addresses the gap

We implement and study **Dgrammar**, a full inference loop that applies grammar masks at the *frontier* only, maintains incremental matcher state with LLGUIDANCE, uses adaptive commit batching, localizes violators, overlaps mask computation with GPU forwards, and instruments timing in the benchmark driver. This directly addresses deterministic alignment to the parser and visibility of constraint overhead. We then add **DPGrammar**, which runs `dp_fix_prefix` on a violated segment to obtain a joint assignment under the current forward, addressing interactions among several committed tokens that greedy remasking fixes only slowly.

1.4 Summary of results and insights

Table **??** summarizes `jsb_medium` runs. Dgrammar clearly improves validity over our LAVE reimplementation on overlapping evaluation settings, and DPGrammar improves further on the same 511 instances as async Dgrammar, with far fewer resamples. Insights: frontier masking plus incremental parsing is a strong deterministic alternative to sampled suffix verification for this task; joint DP repair on violation segments materially reduces repair churn; effective constraint percentage and resample counts are as informative as aggregate wall clock for comparing systems.

2 Related Work

Discrete diffusion and throughput. dLLMs such as LLaDA (**?**) and Dream (**?**) refine tokens in parallel; blockwise accelerators (e.g., DFlash (**?**), Spiffy (**?**)) reduce steps but typically do not guarantee schema compliance by themselves. Our focus is orthogonal: we assume a fixed backbone and study constraint mechanisms.

Constrained decoding in autoregressive models. Tooling such as Outlines (**?**) and XGrammar (**?**) exploits a monotonic prefix for FSM or grammar guided masking. Those interfaces do not transfer directly to diffusion because there is no single causal frontier until we define one.

Constrained decoding for diffusion. **?** formulate CFG constrained diffusion decoding; DINGO (**?**) uses DP style ideas with higher per step cost. LAVE (**?**) samples completions to verify tokens and has become a natural baseline for parallel verification. Plan style decoders (e.g., PVF (**?**)) emphasize structure aware planning; we stay closer to token level masking with an explicit incremental matcher. Sparse trie style acceleration (STATIC (**?**)) motivates keeping transitions sparse; our DP layer explores top K tokens per position rather than the full vocabulary.

Parallel verification and joint decisions. Speculative decoding (**?**) motivates verifying multiple tokens under one forward; LAVE relates in spirit but uses random suffixes. Dgrammar instead commits through a single parser trail; DPGrammar adds a joint Viterbi layer on top when that trail conflicts after a batch unmask.

3 Data

We evaluate on `jsb_medium`, a JSON Schema benchmark split used in our DLLM evaluation harness, aligned with the JSON schema structured output setting discussed in broader benchmarks (**?**). Each instance provides a natural language style prompt, a schema string, and evaluation extracts structured output from the model generation region.

Scale and splits. We report two merge protocols. **LAVE** rows in Table **??** use a four shard Modal run with offsets $[0, 68, 136, 204]$, n=272, seed 0, T=128. **Dgrammar** and **DPGrammar** use nine paired shards with offsets $0, 66, \ldots, 528$, JSONL logs merged by `instance_id`, yielding n=511 unique instances per method. The shard counts differ; we therefore interpret LAVE as the reproduced baseline on the older split and treat the nine shard table as the controlled comparison between Dgrammar and DPGrammar.

Preprocessing. Instances without a usable schema string are skipped in the driver. No additional manual annotation is performed; validity is judged by the benchmark's extraction and schema check, surfaced as the boolean `valid` in result JSON lines.

4 Method

4.1 Baselines reproduced

Our primary external baseline is **LAVE** (**?**), reimplemented to operate with our LLaDA 8B Instruct backbone and `jsb_medium` loader under the same diffusion step budget T=128 where applicable. This isolates the constraint algorithm rather than changing the base model. We additionally compare **Dgrammar** (async greedy, `v2_async_ac4_timed`) against **DPGrammar** (`generate_dp`) on identical instances.

4.2 Dgrammar: proposed deterministic frontier stack

We implement schema constraints with LLGUIDANCE's `LLMatcher`: incremental `try_consume_tokens` reuses state instead of reparsing from scratch.

Constraint at the frontier only. `compute_mask(vocab_size)` is applied at the first unresolved position. Other positions may remain masked; we consume left to right over

revealed runs. This avoids globally masking every row before the block is coherent.

Incremental prefix semantics. Committed non mask spans advance the matcher across denoising iterations without replaying from the prompt on every check.

Adaptive commit (AIMD style). After successful consumes we increase batch size up to $k_{\max}=8$; collisions reset width and identify the first violator.

Violator remask and in place resampling. We remask the violator, zero its logit in the current forward tensor, and iterate alternative argmax choices without always launching a new GPU forward.

Early accept and async overlap. We stop when `is_accepting()` holds. `compute_mask` for the next frontier runs in a CPU thread overlapping the upcoming forward.

4.3 DPGrammar: segment DP on top of Dgrammar

DPGrammar uses the *same* loop as Dgrammar; the only change is repair at violations. When consumption fails at c, we optimize the span $[c, s)$ up to the next mask with `dp_fix_prefix`: Viterbi style DP with DFA keys from `matcher.compute_logit_bias()`, top $K=100$ tokens per position in our runs, merging states and backtracking token replacements. If no feasible path exists, we fall back to Dgrammar's greedy remask. Figure **??** illustrates the shared stack and the DP attachment.

4.4 Completion after generation

Both systems share driver side repair after the diffusion phase: grammar guided greedy autocomplete, masked extension, grammar only closing, and minimal JSON synthesis from the schema when required.

4.5 Implementation and compute

Code builds on PyTorch, Transformers, LLGUIDANCE, and the constrained diffusion CD4d style bindings; benchmarks run on **Modal** with NVIDIA **A100** GPUs in our experiments. Backbone is **GSAI ML LLaDA 8B Instruct**. Block length 32 for timed async runs, remasking `low_confidence`, temperature 0.2 in the driver, seed 0 unless noted. Full hyperparameter tables can be folded into an appendix if space is tight.

4.6 Experimental design and metrics

Schema validity is the primary outcome (binary `valid` per instance). **Wall time** per instance measures end to end driver time. We report median, mean, and P95 to capture heavy tails (LAVE hits a 120 s cap in our run; DPGrammar shows rare long runs). **Effective constraint %** aggregates grammar check time, token selection, mask wait, and autocomplete mask time relative to forward plus overhead, as emitted by `run_dgrammar_timed.py`. **Resamples** counts remasking events. We did not compute formal confidence intervals; differences are large on validity, but future work should add bootstrap CIs on the same split.

5 Results

Table **??** reports aggregate metrics. LAVE reaches 68.8% validity on $n=586$ (nine shards) with median wall time 28.45 s and P95 78.32 s. Dgrammar reaches 84.9% on $n=511$ with median 10.23 s and median effective constraint 20.43%. DPGrammar reaches 99.2% validity on the same 511 instances, median 12.87 s, mean resamples 1.06 versus 32.43 for Dgrammar.

Qualitative observations. Instances where LAVE approaches the timeout often correspond to hard schemas or long generations; Dgrammar avoids sampling variance but can require many greedy resamples when joint structure is wrong, which DPGrammar mitigates. Error modes for DPGrammar on remaining failures include completion phase limits and schemas where minimal JSON fallback is brittle; we leave detailed case studies to appendix space.

‡ LAVE nine-shard evaluation ($n=586$); median and P95 reflect sampling based verification cost.

§ LAVE resamples count lookahead verification draws, not token re-generations; the concept differs from Dgrammar/DPGrammar resamples.

6 Discussion

Interpretation. The gap between LAVE and Dgrammar supports the claim that deterministic frontier parsing with incremental masks is a strong fit for schema constrained JSON under diffusion when sampling based verification is costly or brittle. The gap between Dgrammar and DPGrammar on the same instances isolates the value of joint segment repair beyond greedy remasking.

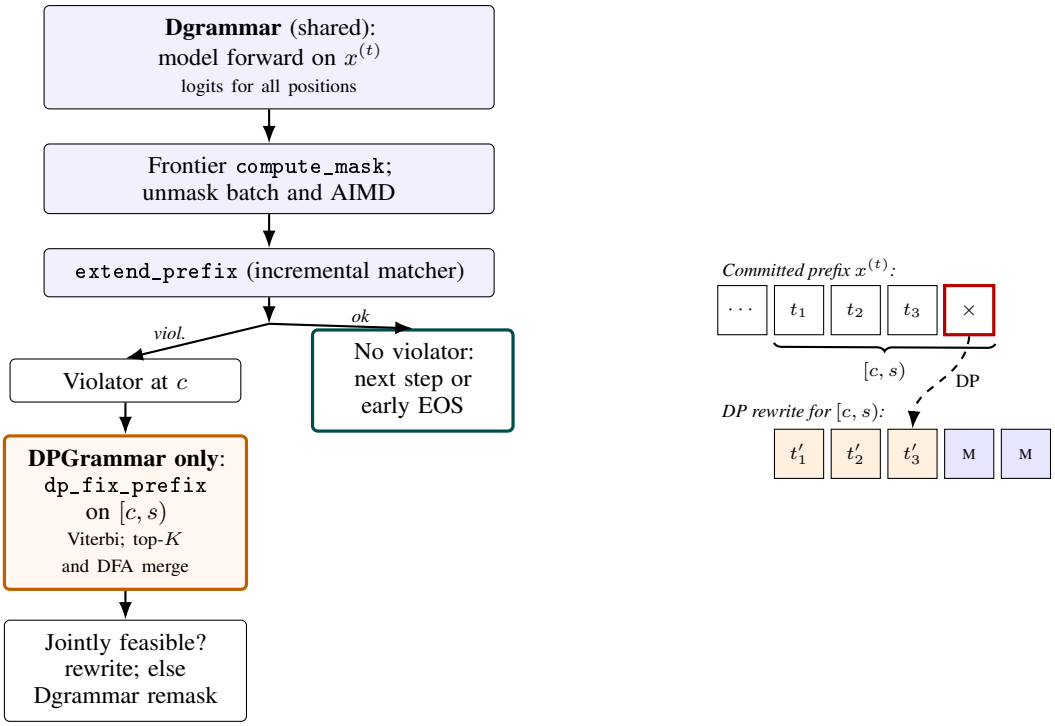

Figure 1: **Dgrammar** (blue tinted): one denoising step applies a full model forward, frontier masking and AIMD unmasking, and incremental `extend_prefix`. **DPGrammar** (orange): when a violator is detected at position c, `dp_fix_prefix` jointly rewrites the span $[c, s)$ via Viterbi over top-K tokens before falling back to greedy remasking.

Method	Validity	Med. time (s)	Mean time (s)	P95 time (s)	Med. eff. const. %	Mean resamples
LAVE ‡ (reimpl., n=586)	68.8%	28.45	34.59	78.32	—	222.6 §
Dgrammar (async, n=511)	84.9%	10.23	13.14	32.10	20.43	32.43
DPGrammar (dp, n=511)	**99.2%**	12.87	14.96	28.55	**1.22**	**1.06**

Table 1: `jsb_medium` results (seed 0, T=128 steps). Dgrammar and DPGrammar are evaluated on the same 511 instances (nine-shard protocol); DPGrammar adds `dp_fix_prefix` on top. LAVE (**?**) uses a nine-shard split covering 586 instances.

Strengths. DPGrammar achieves the highest validity and lowest resample churn in our table; Dgrammar offers a simpler system with strong validity gains over LAVE and competitive medians.

Weaknesses and surprises. Effective constraint % definitions differ in interpretability across methods; DPGrammar's median is low partly because fewer retries occur. Mean and median wall times are close for DPGrammar in our run. DPGrammar validity is below 100%, so completion and fallback stages still matter.

Implications. Practitioners should report forward versus constraint splits, not only total time. For research, joint repair on violation segments is a lightweight add on relative to full sequence DP (**?**).

7 Conclusion and Future Work

We studied grammar constrained decoding for dLLMs with **Dgrammar**, a frontier incremental system engineered for deterministic alignment and timing visibility, and **DPGrammar**, which adds segment level Viterbi repair. On `jsb_medium`, Dgrammar improves substantially over a reproduced LAVE baseline; DPGrammar improves further over Dgrammar on matched instances. **Broader takeaway:** structured generation with diffusion needs both algorithmic ideas and systems style measurement; parser grounded frontiers and joint repair are practical levers.

Future work includes evaluating LAVE on the same nine shard split for a strictly controlled three way comparison; extending to additional JSON splits or code grammars; bootstrap confidence inter-

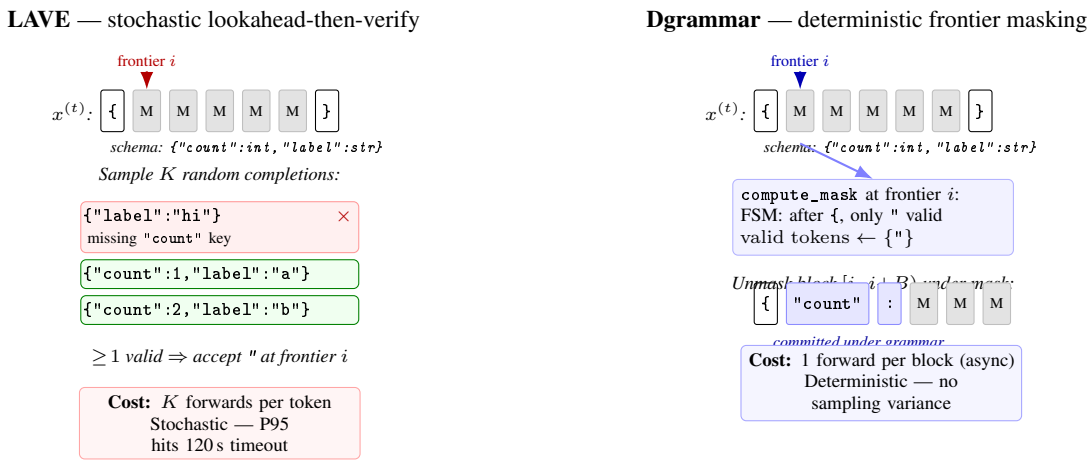

Figure 2: One denoising step under schema `{"count":int, "label":str}`. **LAVE** (**?**) (left) samples K random completions at each frontier token, accepting it only when ≥ 1 suffix is valid—stochastic and $K\times$ more expensive per token, hitting the 120 s per-instance timeout on hard schemas. **Dgrammar** (right) runs `compute_mask` on the incremental FSM state and unmasks an entire block under that mask in one forward pass, overlapped with the next mask computation; the result is deterministic.

vals; and exploring tighter integration with sparse automata acceleration (**?**) for very large vocabularies.

8 Limitations

LAVE and our methods are not evaluated on identical instance sets in Table **??**, so cross column comparisons involving LAVE are indicative rather than strictly controlled. The validity flag depends on benchmark extraction; functional tests beyond schema syntax are not central here. Hyperparameter tuning was limited; different T, block sizes, or temperature could shift outcomes. DPGrammar explores top K tokens per DP position, which can miss globally optimal tokens outside the top K. Compute was cloud A100; other hardware changes overlap benefits.

9 Ethical Considerations

Structured generation can reduce malformed outputs in downstream automation, but *valid JSON is not always semantically safe or fair*: schemas can encode biased or incomplete coverage of user intent. Models and grammars should be reviewed for misuse in high stakes settings (medical, financial). Our benchmark instances originate from public style schema tasks; they may not represent all languages or cultures. Researchers and deployers should combine syntactic guarantees with content moderation and policy controls. We use open backbone weights and public style evaluation data; users of our code should respect model licenses and avoid deploying unreviewed systems in sensitive domains without human oversight.

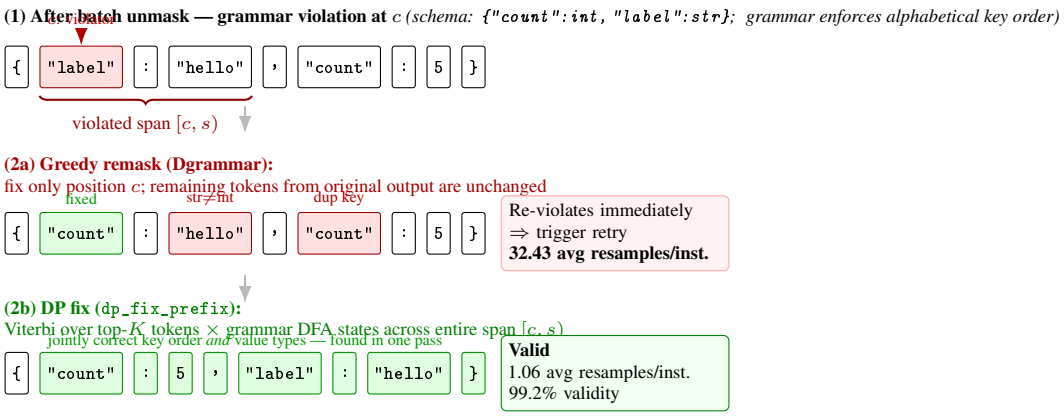

Figure 3: Why DPGrammar lifts validity from 84.9 % (Dgrammar) to 99.2 %. **(1)** A batch unmask produces `"label"` before `"count"`, violating the grammar's alphabetical key-order constraint; the violator at position c spans $[c, s)$. **(2a)** Dgrammar's greedy remask corrects only c: the grammar mask forces `"count"` there, but the value slot still holds `"hello"` (a string in an integer field) and the original `"count"` key remains as a duplicate—causing immediate re-violation and an average of 32.43 resamples per instance (LAVE: 222.6 lookahead draws with similar failure modes on hard schemas). **(2b)** `dp_fix_prefix` runs Viterbi over top-K token candidates across every position in $[c, s)$ simultaneously, jointly finding `"count":5,"label":"hello"`—correct order, correct types, no duplicates—in one pass, reducing to 1.06 avg resamples and 99.2% validity.